

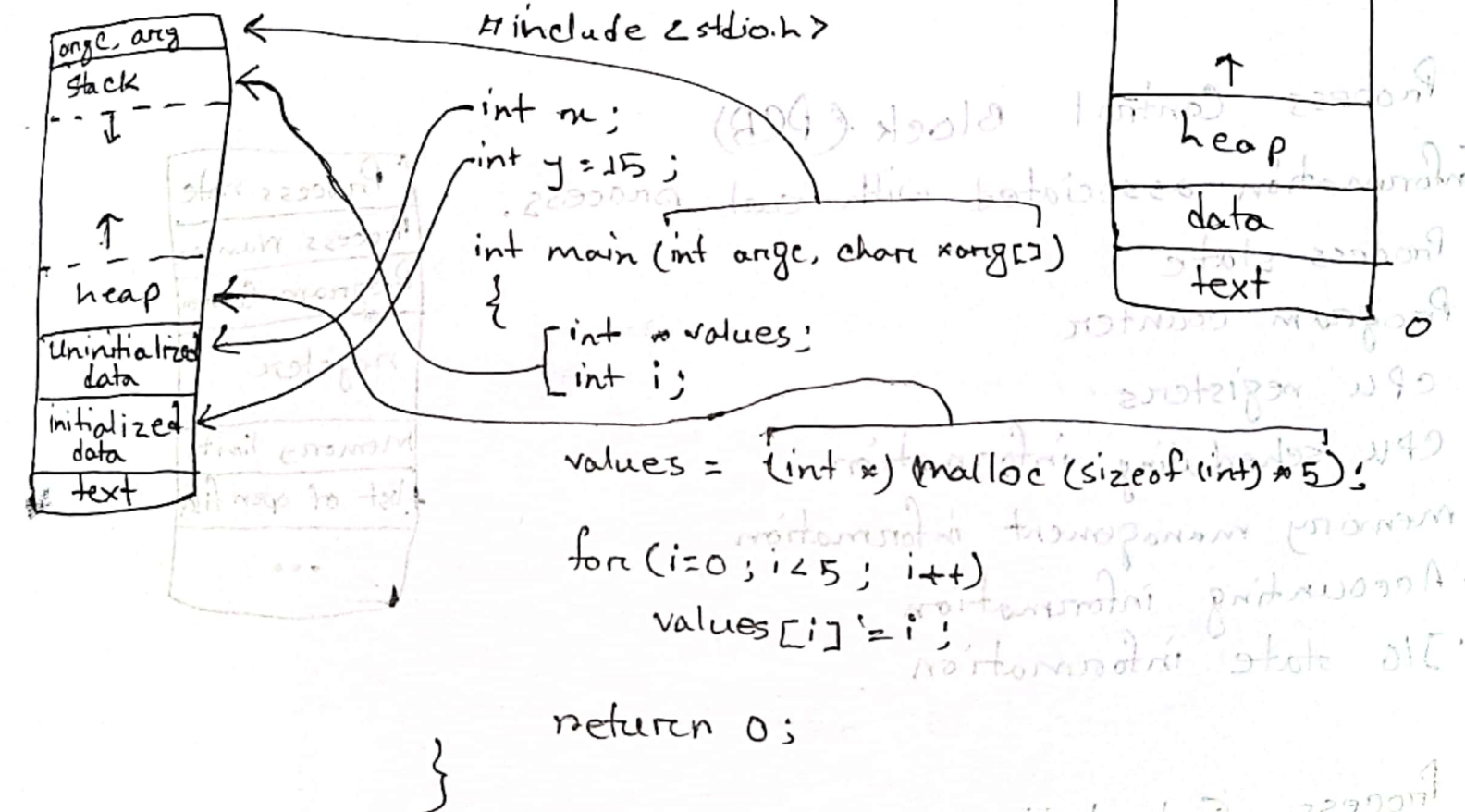
CH-03 : Processes

Process - a program in execution.

Multiple part:

- The program code, also called text section.
- Current activity including program counter.
- Stack containing temporary data.
- Data section containing global variable.
- Heap containing memory dynamically allocated during run time.

Memory Layout of a C program:



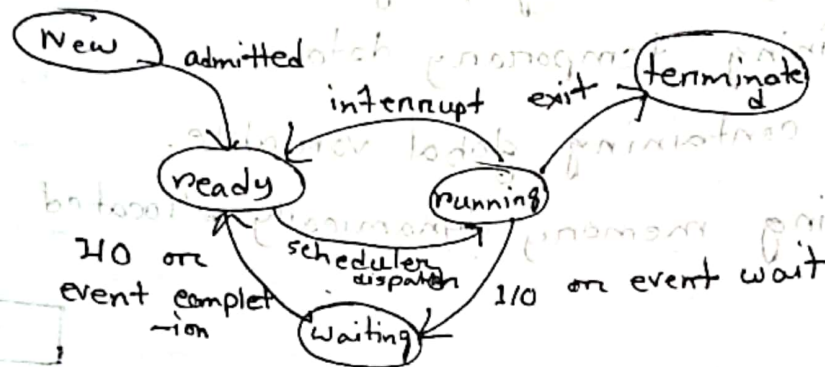
Process State:

As a process executes, it changes state:

New, Running, Waiting, Ready, Terminated

≡

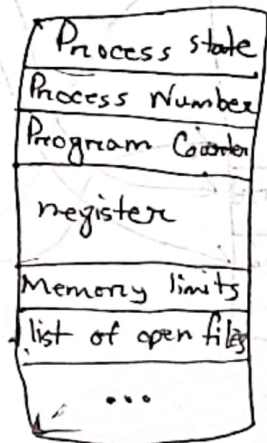
Diagram of Process state:



Process Control Block (PCB)

Information associated with each process.

- Process state
- Program counter
- CPU registers
- CPU scheduling information
- memory management information
- Accounting information
- I/O state information



Process Scheduling:

Process scheduler selects among available processes for next execution on CPU core.

Maintaining scheduling queues of processes

- Ready queue - set of all processes residing in main memory
- wait queues - set of processes waiting for an event.

CPU switch from Process to Process
A context switch occurs when the CPU switches from one process to another.

When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch.

Context-switch time is pure overhead, the system does not useful work while switching.

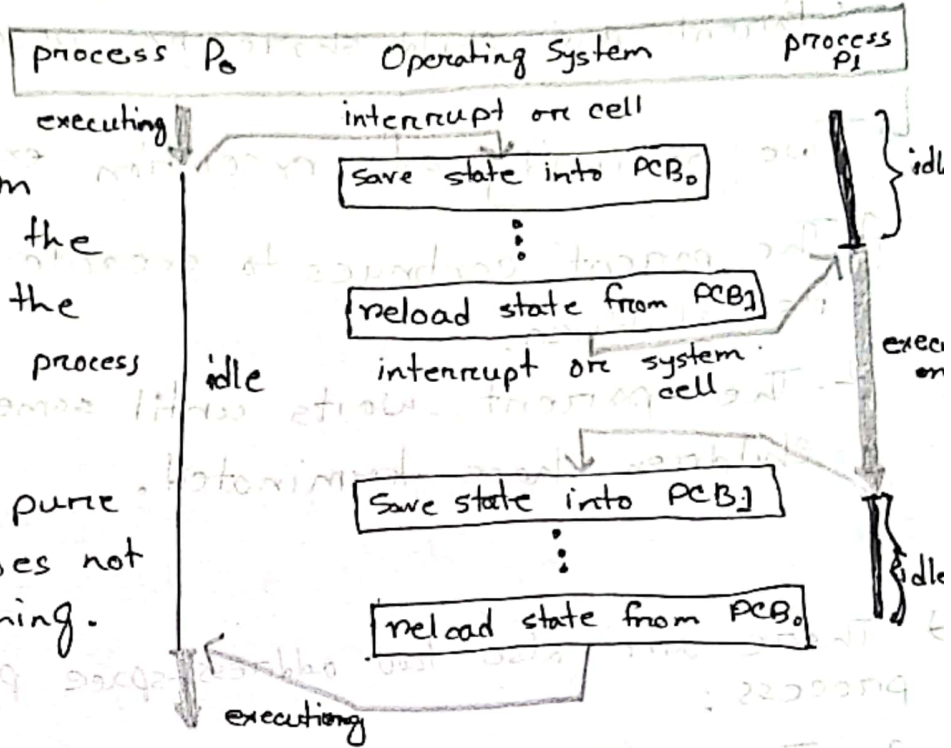


Fig: Context Switching

Multitasking in Mobile Systems:-

System must provide mechanism for

- Process Creation
- Process termination

4 Process creation:

The creating process is called a parent process, and the new processes are called the children of that process.

Parent process create children processes, which, in turn create other processes, forming a tree of processes.

- 1) Resource sharing options:
- Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources.

↳ Two possibility of execution exist:

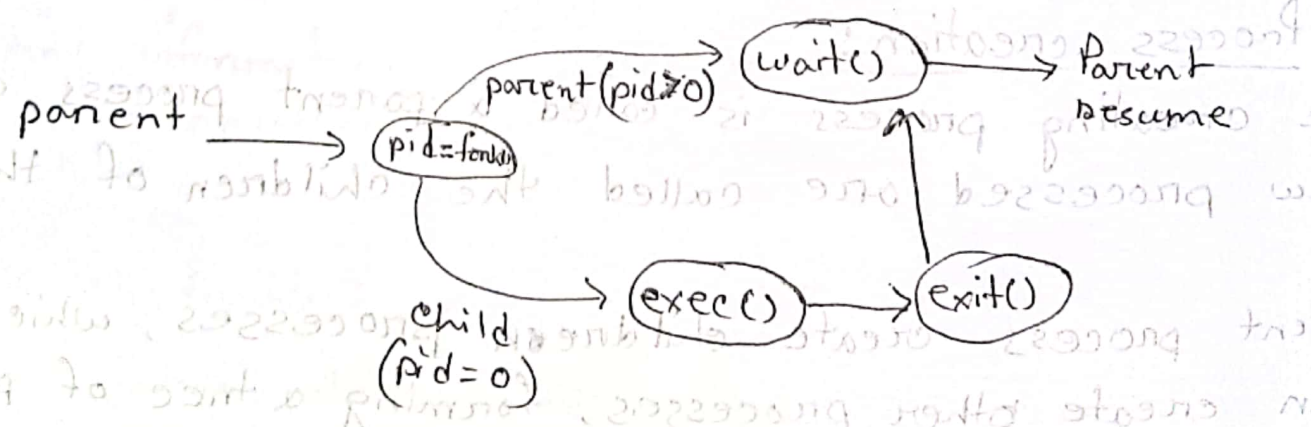
1. The parent continues to execute concurrently with its children.
2. The parent waits until some or all of its children have terminated.

↳ There are also two address-space possibilities for the new process:

1. The child process duplicate of the parent process.
2. The child process has a new program loaded into it.

UNIX example:

fork() ——— exec() ——— wait() ———
 exit() ———



Termination:

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the `exit()` system call.

A parent may terminate the execution of children processes using the `abort()` system call.

1. child has ~~exceed~~ exceeded allocated resources.
2. Task assigned to child is no longer required.
3. The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.

Interprocess Communication:

Process within a system may be independent or cooperating.

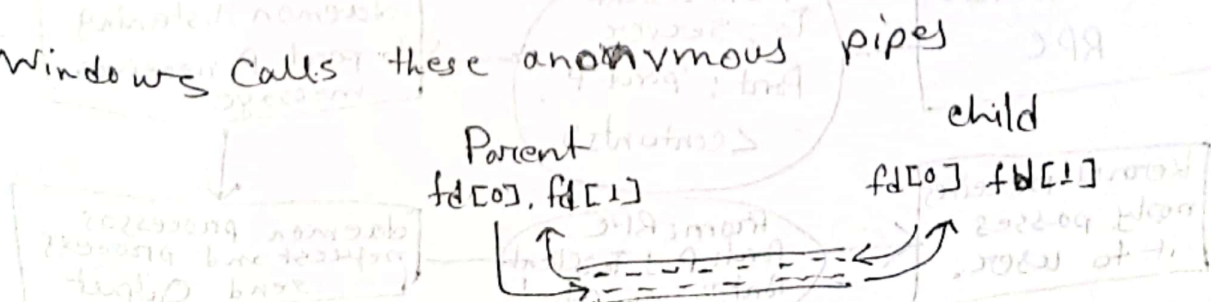
Cooperating process need interprocess communication.

(IPC). It has two models:

- ① Shared memory $\equiv$
- ② Message passing $\equiv$

Ordinary pipes; allows communication in standard producer-consumer style.

Windows calls these anonymous pipes



Socket: - endpoint for communication.

Java provides ~~two~~ three different types of sockets:

① Connection-Oriented (TCP) ② Connectionless (UDP) ③ MulticastSocket

Remote Procedure calls (RPC):

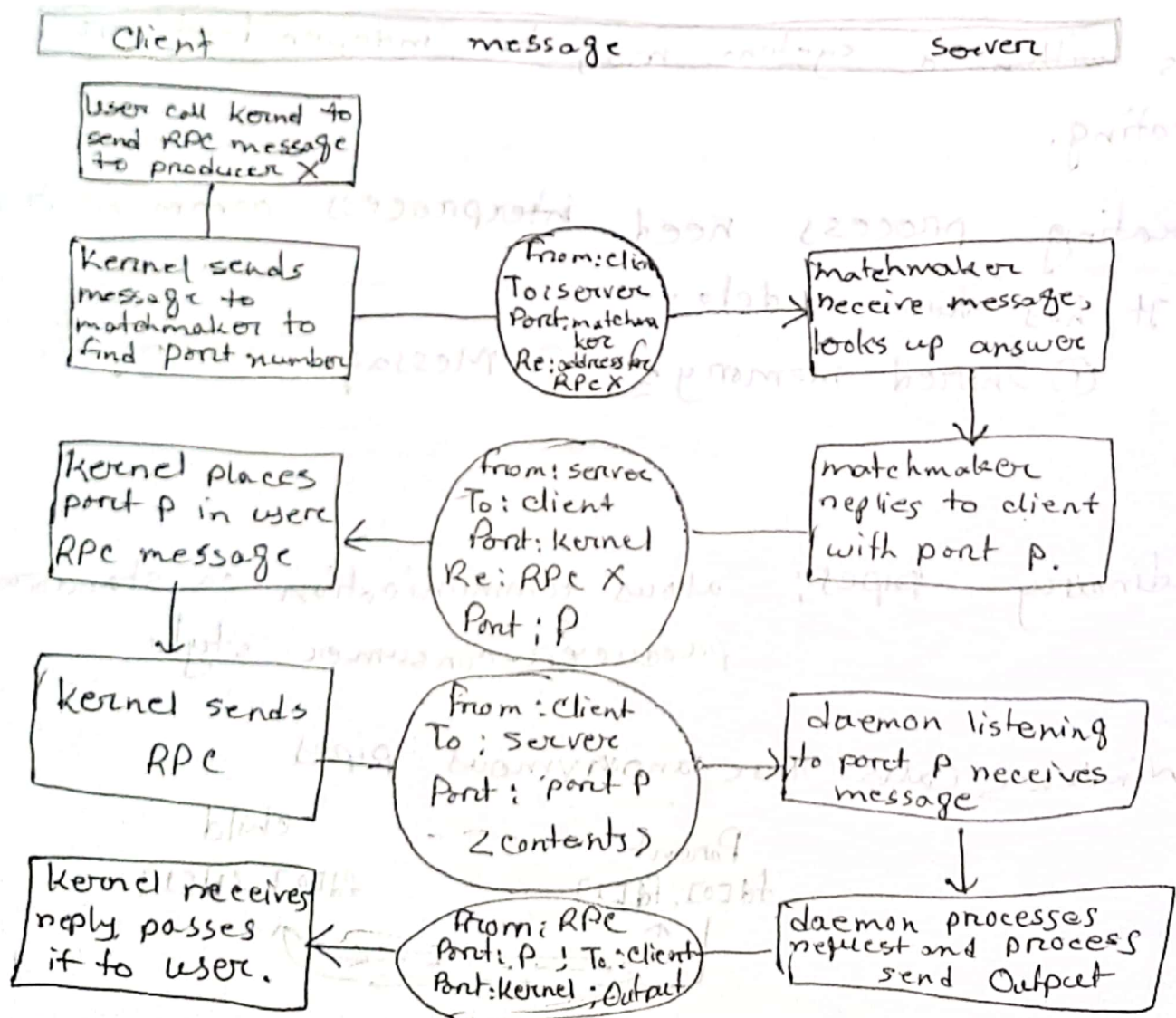
socket
192.168.25.2:1425

socket
192.168.25.2:1425

RPC was design as a way to abstract the procedure-call mechanism for use between systems with network connections.

On windows systems, stub code is compiled from a specification written in the Microsoft Interface Definition Language (MIDL), which is used for defining the interface between client and server programs.

Execution of a remote procedure call (RPC)



Ex
3.9 Three major complications that concurrent processing adds to an operating system:

Concurrent processing introduces several complications to an OS, particularly in areas like resource management, synchronization and communication. Here are three major complications that concurrent processing adds to an OS.

1. Race Conditions:

- Occur when processes access shared resources concurrently without synchronization.
- Can lead to inconsistent results in critical sections.
- Solved by using synchronization mechanism like mutexes, semaphores or locks.

2. Deadlock :

- Happens when processes are stuck waiting for resources held by each other.
- Causes system performance degradation or freezes.
- Prevented or managed via deadlock prevention, avoidance, detection or resource ordering.

3. Resource starvation:

- Occurs when lower-priority processes are consistently denied resources.
- Leads to indefinite delays and system instability.
- Solved by using fair scheduling algorithms like round-robin or priority aging.

3.4 Context switch in 2 cases —

↳ When a context switch occurs and the new context is already in one of the available register sets, the CPU can switch to that register set instantly, minimizing overhead.

↳ If the new context is in memory and all register sets are in use, the system must save one of the current register sets to memory and load the new context into the freed register set, which adds additional overhead due to these save/load operation.

* / use previous fig * /

A context switch occurs when the CPU switches from one process to another.

When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context-switch. Context of a process represent B in the PCB.

2.8 The action taken by a kernel to context-switch between processes.

A context switch occurs when the CPU switches from one process to another.

/x Draw previous fig x/

Here's a detailed overview of the action taken by a kernel during a context switch:

1. Trigger: An event like a timer interrupt or I/O completion initiates the need for a context-switch.
2. Save State: The kernel saves the current state of the CPU and associated memory-management of the running process into its Process Control Block (PCB).
3. Select Process: The scheduler determines which process to run next based on criteria like priority or fairness.
4. Load state: The kernel loads the state of the next process from its PCB, setting up the CPU and memory configuration for this process.
5. Resume Execution: Control is handed over to the new process, allowing it to continue where it left off.

These steps ensure that the system handles multiple processes efficiently, maintaining system responsiveness and stability.

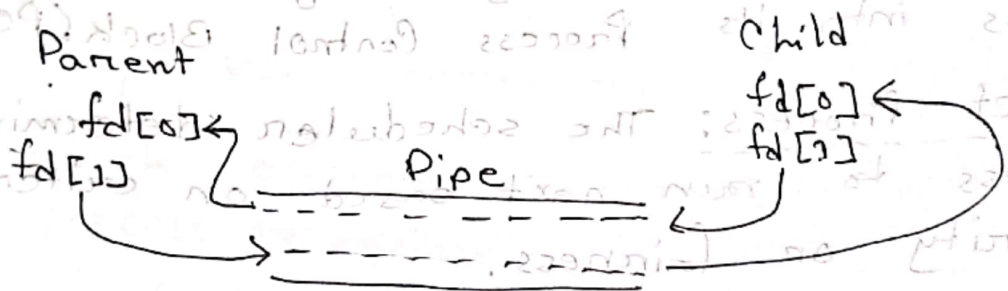
3-19 | ≡

Example where ordinary pipes are more suitable:

⇒ Inter-process communication (IPC) between related processes (parent & child):

▫ Scenario - A parent process spawns a child process and needs to send data to it.

▫ Why suitable - Ordinary pipes are perfect for this situation because they provide unidirectional communication between a parent and child process, and since both processes share the pipe automatically no need for a filename or explicit connection.



Example where named pipes are more suitable.

⇒ Communication between unrelated or independently running processes.

▫ Scenario: Two processes, running at different times or start independently, need to communicate.

▫ Why suitable: Named Pipes (FIFOs) allow unrelated processes to communicate through a file in the filesystem, which can be accessed by different processes as long as they know the name of the pipe.

- 3.17 ✓ Synchronous VS Asynchronous Communication
- ✓ Automatic Buffering VS Explicit Buffering
 - ✓ Send by Copy VS Send by Reference
 - ✓ Fixed sized message VS Variable-sized message.

need to
convert this